0091. Decode Ways

n | Category: 1-D Dynamic Programming | LeetCode: https://leetcode.com/problem

Problem

- A=1 … Z=26. Count the ways to decode a digit string. "06" is not valid ("0" has no letter).

Intuition

- Dp[i] = ways to decode the first i characters.
- One-digit step valid if s[i-1] != '0'; two-digit step valid if s[i-2..i-1] is 10..26.

Brute Force

- Recursion trying 1 and 2 digits: O(2^n).

Optimal Approach

- Dp[0] = 1.
- Dp[i] = (s[i-1] != '0' ? dp[i-1] : 0) + (10 <= two-digit <= 26 ? dp[i-2] : 0).
- Roll into two variables.

Worked Example

```
"226".

dp[0]=1
dp[1]: "2"          -> 1
dp[2]: "2","22"     -> 1 + 1 = 2
dp[3]: "6","26"     -> 2 + 1 = 3   (BZ, VF, BBF)
```

Java Solution

```java
public class DecodeWays {
    public int numDecodings(String s) {
        int prev2 = 1, prev1 = s.charAt(0) == '0' ? 0 : 1; // dp[0], dp[1]
        for (int i = 2; i <= s.length(); i++) {
            int cur = 0;
            if (s.charAt(i - 1) != '0') cur += prev1;
            int two = (s.charAt(i - 2) - '0') * 10 + (s.charAt(i - 1) - '0');
            if (two >= 10 && two <= 26) cur += prev2;
            prev2 = prev1;
            prev1 = cur;
        }
        return prev1;
    }
}
```

Complexity

- Time: O(n). Space: O(1).

Pitfalls

- Leading zero → 0. "10" and "20" have exactly one decoding; "30" has none.